Convolutional Neural Networks

Fundamentals · Filters · Pooling · Feature Maps · CIFAR-10

Lecture 7

Week 8 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda

Lecture 7

Five sections · CIFAR-10 running example

01 **Why CNNs?**
FC problems · invariance · equivariance

02 **Convolution Operations**
1D · 2D · kernel · stride · padding · dilation

03 **Channels & Receptive Fields**
Feature maps · multi-channel · receptive field growth

04 **Pooling & Downsampling**
Max pool · GAP · upsampling · transposed conv

05 **CNN in Keras**
Conv2D · MaxPool2D · CIFAR-10 · CNN vs MLP comparison

Dataset: keras.datasets.cifar10 · Géron Ch. 14 pp. 543–592 · Prince (2023), "Understanding Deep Learning", Ch. 10 pp. 161–185.

Lecture 6 → Lecture 7

Lecture 6 — Data Side

✓ tf.data pipeline: map → cache → shuffle → batch → prefetch

✓ Normalization: ÷255 vs z-score per channel (SGD)

✓ Augmentation as implicit regularization

✓ Keras Normalization layer baked into the model

Lecture 7 — Architecture Side

→ Why CNNs? Problems with fully-connected on images

→ Convolution: kernel · stride · padding · dilation

→ Channels · feature maps · receptive fields

→ Pooling · downsampling · transposed convolution

→ Build and train a CNN on CIFAR-10 in Keras

Good data pipelines (L6) + the right architecture (L7) = the full recipe.

01

Why CNNs?

3 problems with FC · invariance · equivariance

Why Not Fully-Connected on Images?

Problem 1 — High-dimensional inputs

A 224×224 RGB image has 150,528 features. One FC hidden layer of equal size needs > 22 billion weights.

Problem 2 — No notion of "nearby"

FC treats every pixel equally. FC layers do not encode spatial adjacency; they treat pixels as a flat vector unless the model learns structure from data.

Problem 3 — No translation stability

Shifting a cat by 3 pixels changes all 150k inputs. FC must re-learn the cat at every position — wildly inefficient.

Solution: Convolutional layers — local connections + same weights reused at every position

Invariance and Equivariance

Invariance

$$f[t[x]] = f[x]$$

- The output remains the same even when the input is transformed.
- Important for **image classification**: networks should recognise objects regardless of translation, rotation, flipping, or warping.

Both images are classified as "mountain" regardless of the horizontal shift.

Equivariance

$$f[t[x]] = t[f[x]]$$

- The output changes in the same way as the input under the transformation.
- Important for **image segmentation**: each transformed input should produce a similarly transformed output.

The colored overlay translates equivalently with the input, maintaining equivariance.

Figures adapted from Prince (2023).

02

Convolution Operations

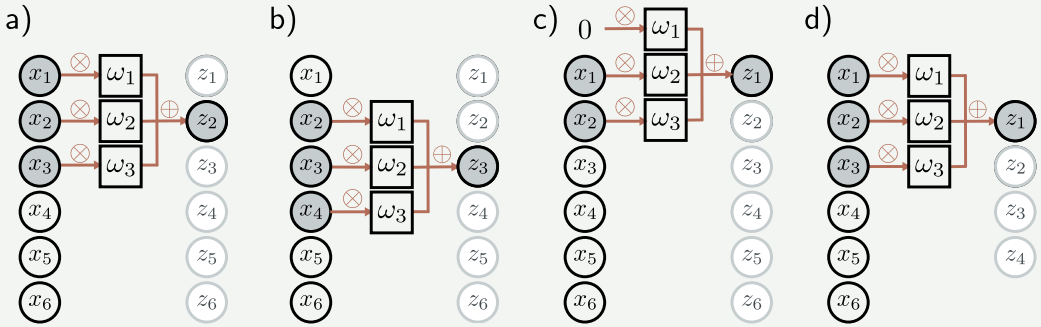
1D intuition · 2D images · kernel · stride · padding · dilation

1D Convolution — Core Idea

- A **convolutional layer** applies the same weights (a **kernel/filter**) across the input.
- Each output is a weighted sum of nearby inputs.
- The same kernel slides across the full input: this is weight sharing.
- Convolution is **translation equivariant**: shifting the input shifts the output.

Kernel size 3:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$



a) Kernel covers x_1 to x_3 and computes one output: $z_2 = \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3$

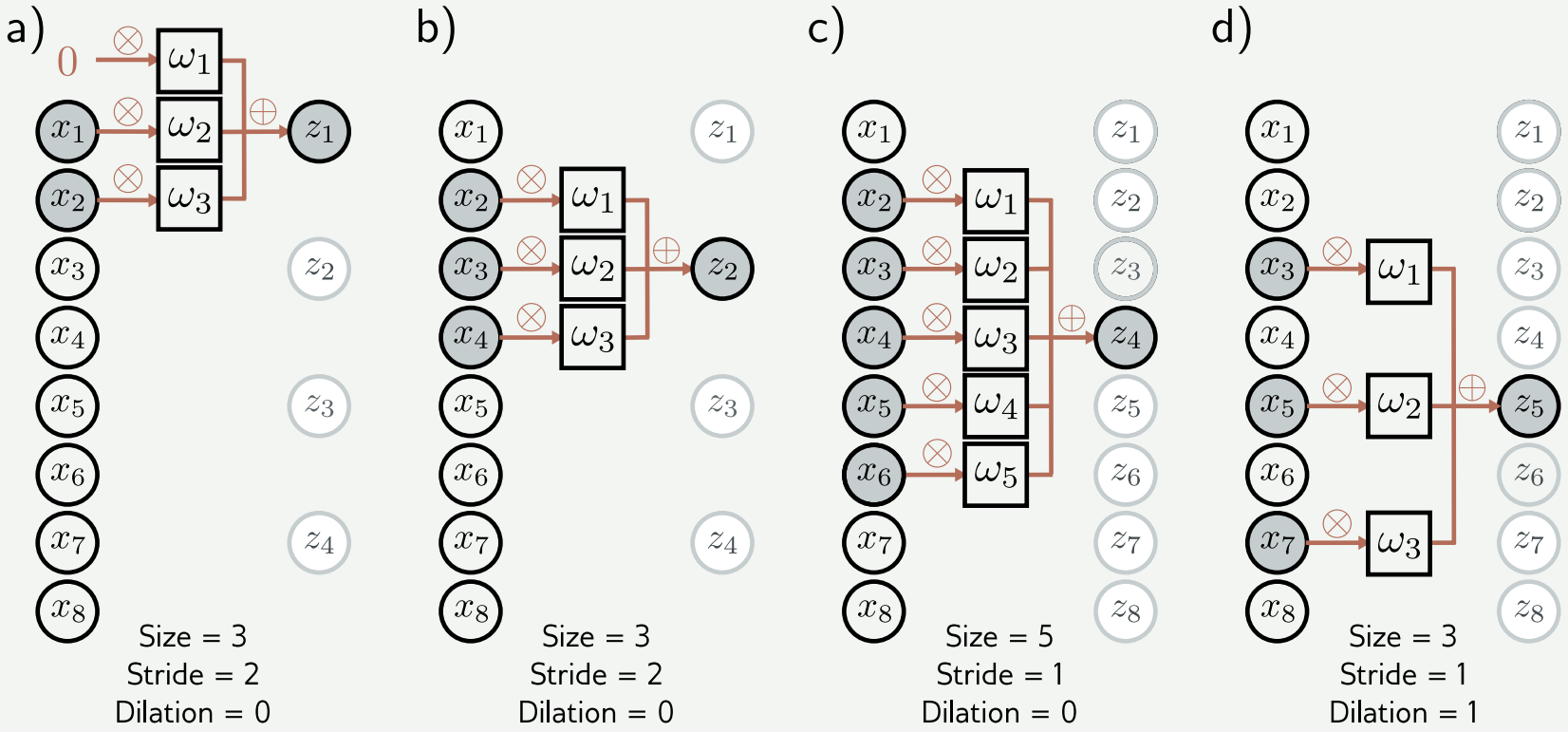
b) Same kernel shifts one step to compute the next output.

c) Boundary outputs require values outside the input; **zero-padding** handles this.

d) Valid convolution skips boundary positions, giving a smaller output.

Figures adapted from Prince (2023).

Stride, Kernel Size, and Dilation



In Keras terminology, ordinary convolution has dilation_rate=1. Dilation means dilation_rate > 1

Figures adapted from Prince (2023).

Padding and Output Dimensions

padding='same' (zero-padding)

- Output size: $\lceil H / S \rceil$ (for stride S, kernel K)
- Pads edges so the kernel centres on boundary pixels.
- Not the Keras default but commonly used to preserve spatial dimensions.
- Example: H=32, K=3, S=1 → output 32 | S=2 → output 16

padding='valid' (no padding)

- Output size: $\lfloor (H - K) / S \rfloor + 1$
- Keras Conv2D default. Only positions where the kernel fits inside the input.
- Useful for precise size reductions.
- Example: H=32, K=3, S=1 → output 30 | S=2 → output 15

General formula (given explicit P, assuming dilation rate = 1): $\lfloor (H + 2P - K) / S \rfloor + 1$ e.g. $(32 + 2\times1 - 3)/1 + 1 = 32$

2D Convolution for Images

Extend 1D to 2D: kernel slides horizontally and vertically

3×3 kernel:

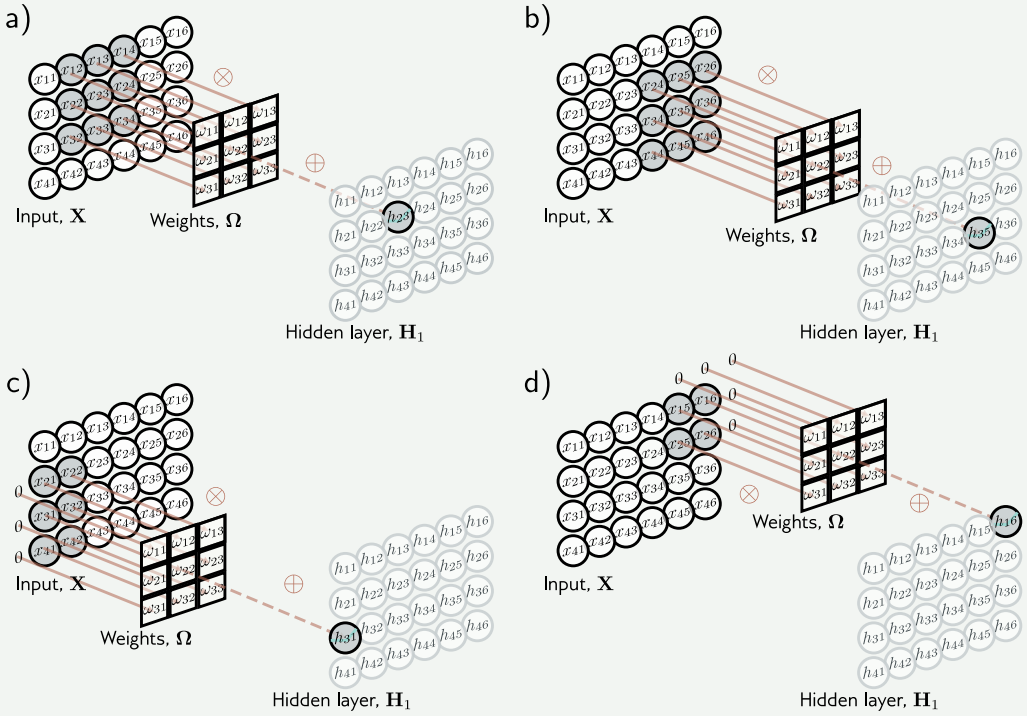
$$h_{ij} = a\left[\beta + \sum_{m=1}^{3}\sum_{n=1}^{3}\omega_{mn}x_{i+m-2,j+n-2}\right]$$

Each output position depends on a 3×3 patch of the input.

RGB input (3 channels) and a 3×3 kernel => 3×3×3 = 27 weights.

The kernel covers all C_in channels simultaneously → one output channel.

To produce C_out channels we need: K^2 × C_in × C_out weights + C_out biases.



Figures adapted from Prince (2023).

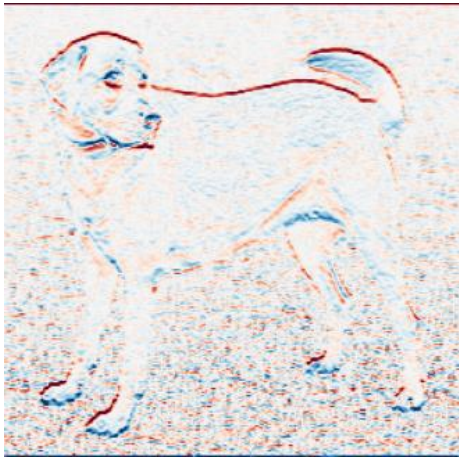
Manual Filters: What Convolution Computes

Hand-designed filters reveal what convolution computes. CNNs learn these from data.

Manual convolution with hand-designed filters — same operation as a learned Conv2D kernel

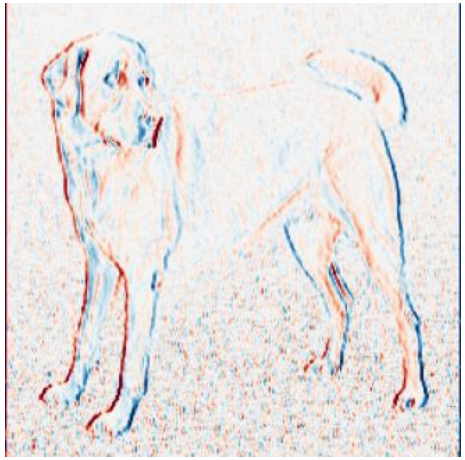
Original image

Horizontal edge (Sobel-y)



```
[ -1  -2  -1 ]
[  0   0   0 ]
[  1   2   1 ]
```

Vertical edge (Sobel-x)



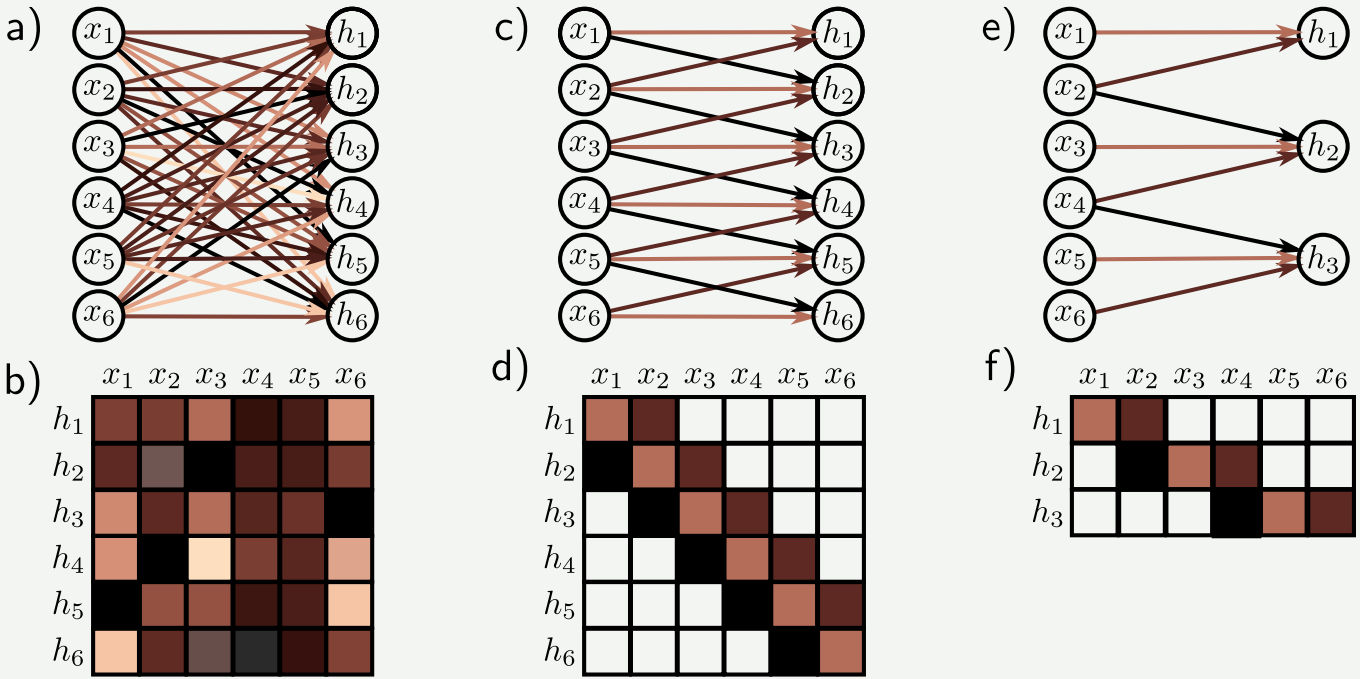
```
[ -1   0   1 ]
[ -2   0   2 ]
[ -1   0   1 ]
```

Sharpening filter

```
[  0  -1   0 ]
[ -1   5  -1 ]
[  0  -1   0 ]
```

Conv2D(32, 3) on CIFAR-10: 896 params · Dense layer producing same output volume: 100.7M params → 112,384× more · Weight sharing is the key!

Fully Connected vs. Convolutional — Weight Sharing



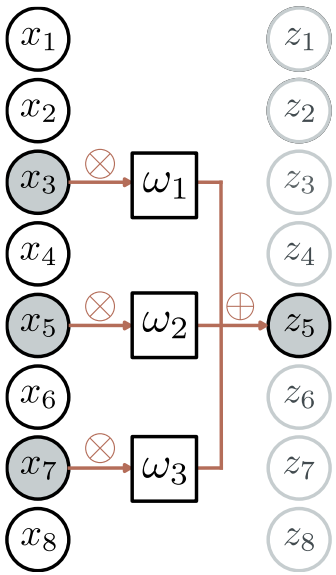
FC layers use dense connections between inputs and hidden units (a–b), while CNN layers use shared local weights that create sparse, structured matrices (c–d). Changing convolution settings, such as stride 2 (e), changes the sparse pattern, but it remains a special case of a FC network (f).

The same local pattern detector is reused everywhere in the image. Conv is FC with zero-masked and shared weights.

Figures adapted from Prince (2023).

Exercise — 1D Convolution

The figure shows a 1D convolution transforming $\mathbf{x} = [x_1, x_2, \ldots, x_8]$ with kernel $\mathbf{w} = [w_1, w_2, w_3]^T$ to produce $\mathbf{z} = [z_1, z_2, \ldots, z_8]$. The highlighted computation produces z_5.



Given $x = [1,2,3,4,5,6,7,8]$ and $w = [-1,2,1]^T$

i. Kernel size = _____

ii. Stride = _____

iii. Dilation rate = _____

iv. Padding type = _____

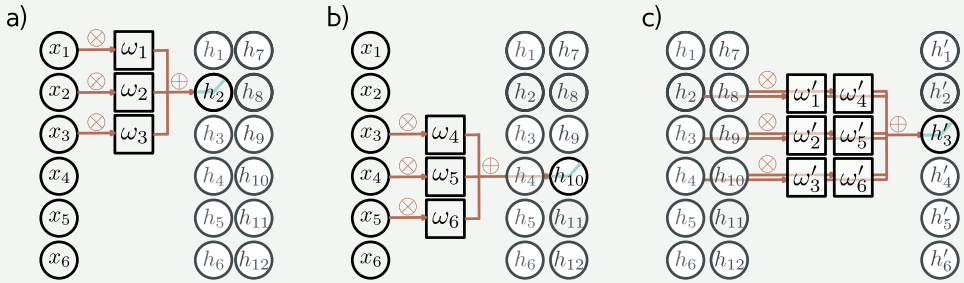
v. z_2 = _____

03

Channels and Receptive Fields

Feature maps · multi-channel · receptive field growth · 1×1 convolution

Channels and Feature Maps



Figures adapted from Prince (2023).

Why multiple channels?

A single convolution can lose information: local averaging mixes nearby inputs, and ReLU clips negative responses to zero.

One kernel → one feature map

Each kernel produces one output channel. Different kernels can capture different patterns in the same input.

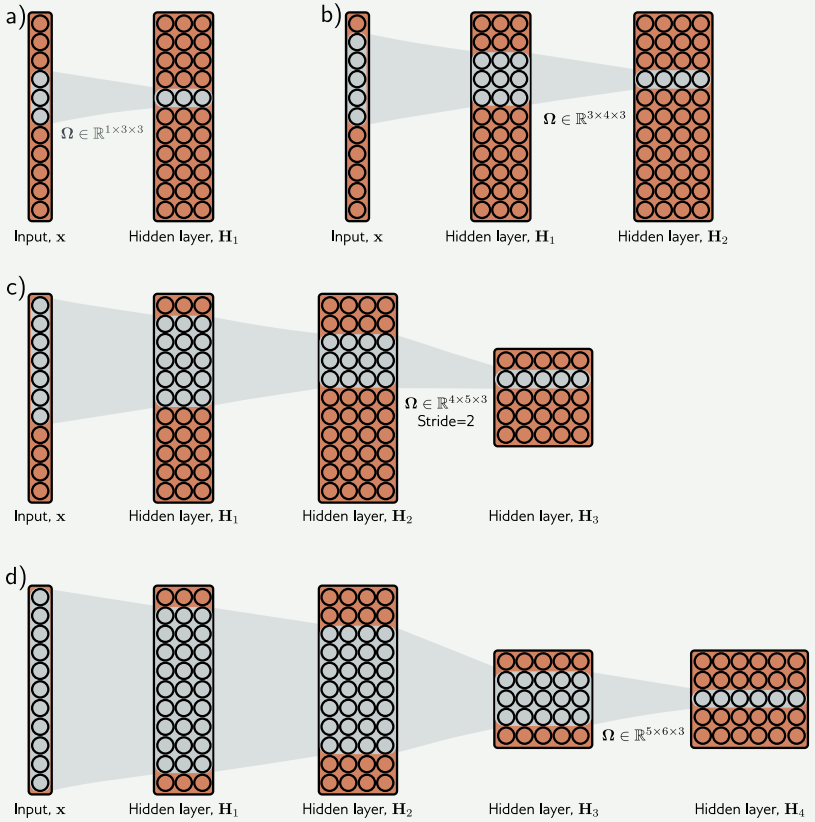
Stack feature maps

Applying C_out kernels in parallel gives C_out output channels — a feature-map tensor.

Channel rule

Each output channel comes from one learned filter. A layer with **C_out output** channels learns **C_out filters**, and every filter combines info from all **C_in input channels**.

Receptive Fields — How Depth Expands Context



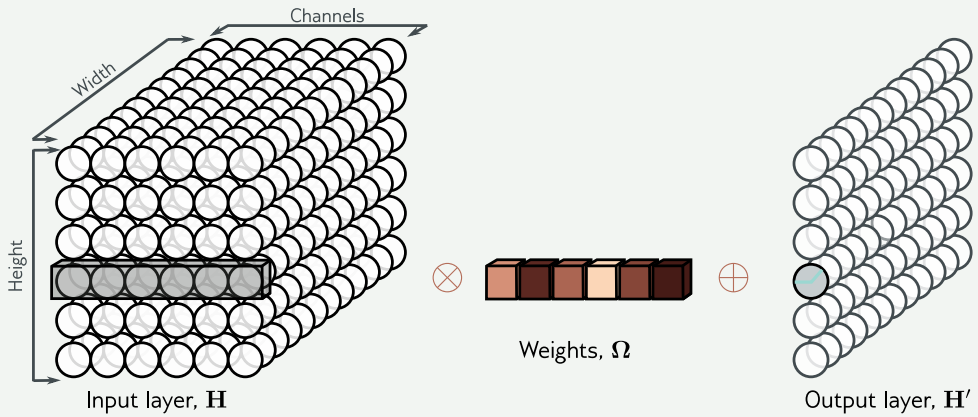
- The receptive field of a hidden unit in the network is the region of the original input that feeds into it.
- In CNNs, the receptive field grows with depth as information is aggregated from more input positions.
- Example:
  - First layer (kernel size 3): RF size = 3
  - Second layer (kernel size 3): each unit depends on 3 values from layer 1 → total RF = 5
- Deeper layers integrate broader context from the input.
- This gradual expansion allows CNNs to capture both local and global patterns.

Figures adapted from Prince (2023).

Takeaway: deeper layers see wider context; stride/pooling makes later layers cover more input pixels per layer.

1×1 Convolution — Channel Mixing

- Sometimes we want to change the number of channels between layers without altering spatial dimensions
- This is achieved using a **1 × 1 convolution**
- At each spatial position, we take a weighted sum across all input channels
- The kernel size is **1 × 1**, and the weights have shape 1 × 1 × C_in × C_out
- This operation produces C_out output channels from C_in input channels
- Equivalent to applying a small fully connected network at each spatial location



Figures adapted from Prince (2023).

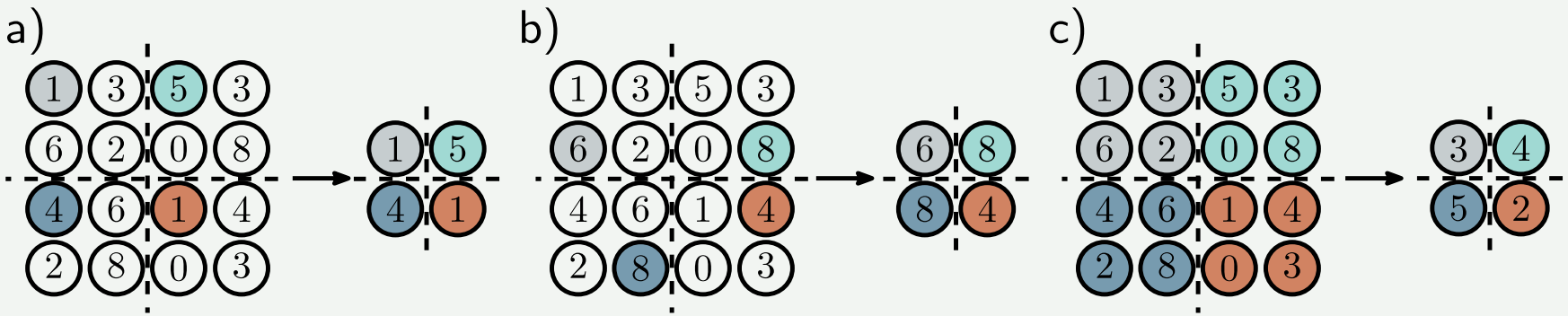
Combined with bias + activation → nonlinear channel remixing. Used in ResNet bottleneck blocks (L8).

04

Pooling and Downsampling

Max pooling · global average pooling · transposed convolution · upsampling

Downsampling — Three Approaches



a) Sub-sampling (stride 2): skip every other position *b) Max Pooling: keep max of 2×2* *c) Mean/Average Pooling: average 2×2*

Figures adapted from Prince (2023).

For even H and W with stride 2, these operations halve spatial dimensions while preserving channels. Applied independently per channel. Keras MaxPool2D default padding='valid'.

MaxPooling and Global Average Pooling

Max Pooling (spatial downsampling)

Slides a 2×2 window with stride 2; keeps the maximum.

Partial translation invariance: shifting input by 1px, many max values unchanged.

No learnable parameters. Applied per channel.

Output: about H/2 × W/2 × C when H and W are even (Keras uses floor division).

Keras: MaxPooling2D(pool_size=2, padding='valid')

Global Average Pooling (spatial collapse)

Computes the mean of each entire feature map → one number per channel.

Output: vector of length C. Zero parameters.

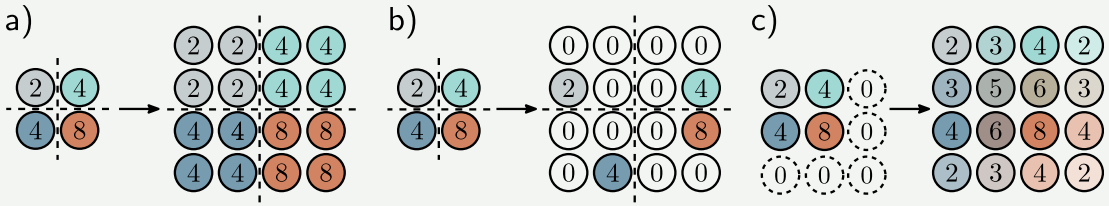
Collapses explicit spatial layout into channel-level averages → implicit regularizer.

More robust to image size variation than Flatten+Dense.

Often preferred in modern CNN classifiers (ResNet, EfficientNet).

Keras: MaxPooling2D(pool_size=2) · GlobalAveragePooling2D()

Upsampling and Transposed Convolution



a) **Duplication**: Duplicate each pixel (and its channel values) to upscale by a factor.

b) **Max Unpooling**: Used after max pooling. Values are placed back at their original max locations.

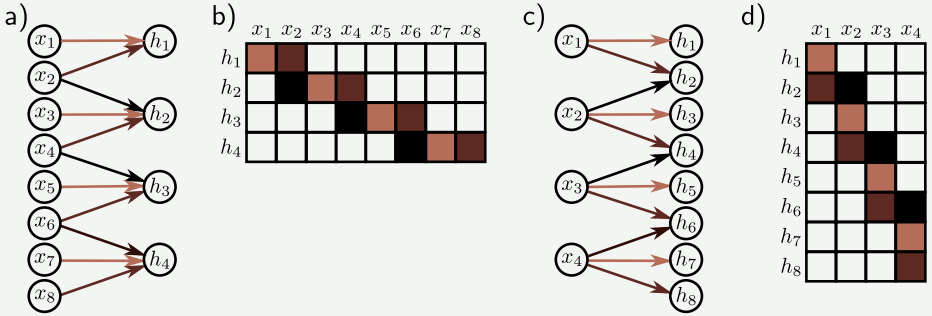
c) **Bilinear Interpolation**: Fill in missing values by interpolating between known samples.

a) Downsampling with kernel size 3, stride 2, and zero-padding. Each output is a weighted sum of 3 inputs.

b) This can be expressed by a weight matrix (same color indicates shared weight).

c) In transposed convolution, each input contributes 3 values to the output layer, which has twice as many outputs as inputs.

d) The associated weight matrix is the transpose of that in (b).



Figures adapted from Prince (2023).

With suitable stride/kernel/padding settings, these operations can double spatial dimensions. Used in segmentation decoders (U-Net, L11) and generative models (GANs, Diffusion — L20–L21).

05

CNN in Keras

Conv2D · MaxPooling2D · CIFAR-10 · CNN vs MLP comparison

The CNN Building Block

Standard block: Conv → BN → ReLU → Pool

Conv2D(F, 3, padding="same") — extracts local features

BatchNormalization() — stabilizes activations (L4)

Activation("relu") — introduces non-linearity

MaxPooling2D(2) — halves spatial dims

Repeat 3–5 blocks, doubling filters: 32 → 64 → 128

GlobalAveragePooling2D() — collapses spatial dims

Dense(num_classes, activation="softmax")

Keras Conv2D key parameters

filters: number of output channels (32, 64, 128…)

kernel_size: 3 (standard) or (3, 3)

padding: "same" keeps size / "valid" shrinks it

strides: default 1; some modern nets use 2 instead of MaxPool

kernel_initializer: "he_normal" with ReLU (L4)

activation: set None → apply BN + Activation separately

Rule: double the filters every time you halve the spatial dimensions.

Conv2D and MaxPooling2D — Keras API

Key layer signatures

Conv2D(filters, kernel_size, padding, kernel_initializer, activation) · MaxPooling2D(pool_size, strides)

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import BatchNormalization, Activation, GlobalAveragePooling2D

# Standard conv block helper
def conv_block(filters):
    return [Conv2D(filters, 3, padding='same', kernel_initializer='he_normal'),
            BatchNormalization(), Activation('relu'), MaxPooling2D(2)]

# Check output shape
import tensorflow as tf
x = tf.random.normal([1, 32, 32, 3])
layer = Conv2D(32, 3, padding='same')
print(layer(x).shape)   # → (1, 32, 32, 32)  same spatial, 32 channels

layer_valid = Conv2D(32, 3, padding='valid')
print(layer_valid(x).shape)  # → (1, 30, 30, 32)  shrinks by K-1=2
```

Tip: always print layer output shapes when stacking blocks to catch dimension errors early.

Full CNN on CIFAR-10 — Baseline Model

3-block CNN: input 32×32×3 → 4×4×128 → class probabilities

Notebook uses an equivalent Functional API version to extract feature maps later.

```
model = Sequential([
    Input(shape=(32, 32, 3)),
    # Block 1: 32×32×3  →  16×16×32
    Conv2D(32, 3, padding='same', kernel_initializer='he_normal'),
    BatchNormalization(), Activation('relu'), MaxPooling2D(2),
    # Block 2: 16×16×32  →  8×8×64
    Conv2D(64, 3, padding='same', kernel_initializer='he_normal'),
    BatchNormalization(), Activation('relu'), MaxPooling2D(2),
    # Block 3: 8×8×64  →  4×4×128
    Conv2D(128, 3, padding='same', kernel_initializer='he_normal'),
    BatchNormalization(), Activation('relu'), MaxPooling2D(2),
    GlobalAveragePooling2D(),        # 4×4×128 → 128
    Dense(10, activation='softmax') # 128 → 10 classes
])
model.compile(optimizer=AdamW(1e-3, weight_decay=1e-4, clipnorm=1.0),
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

~95k parameters · ~76.8% best val / 76.0% test in this run on CIFAR-10 (Colab T4, no augmentation · best epoch 65 (early stopping))

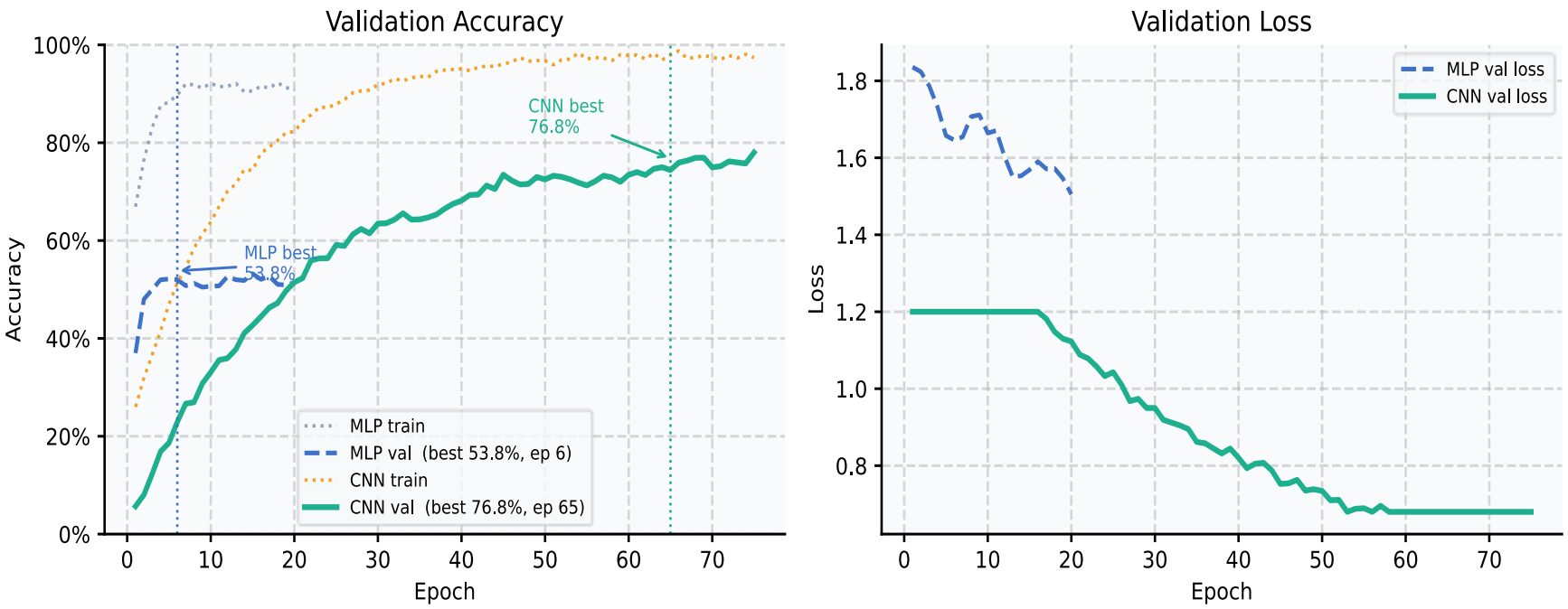
CNN vs MLP — CIFAR-10 Comparison

	MLP (L5 baseline)	3-block CNN (this lecture)
Architecture	Flatten → Dense(256) → Dense(128) → Dense(10)	**Conv×3 → GAP → Dense(10)**
Parameters	822,410 parameters	**95,434 parameters (8.6× fewer)**
Val Accuracy	53.8% (AdamW, ep 6)	**76.8% (AdamW, ep 65)**
Training time	~3 min (early stop ep 6)	**~15 min (early stop ep 65)**
What it ignores	Spatial structure completely	**Global context requires sufficient receptive field/depth before global aggregation**
Key advantage	Simple, fast, interpretable	**Spatial features, weight sharing**

CNN achieves +23.0pp validation accuracy with 8.6× fewer parameters. Same optimizer (AdamW), z-score, and callbacks — the architecture is the key difference. Full-run Colab T4 result. FAST-mode notebook gives slightly lower numbers.

CNN vs MLP — CIFAR-10 Training Curves

CNN vs MLP — CIFAR-10 (AdamW · z-score · EarlyStopping + ReduceLROnPlateau)



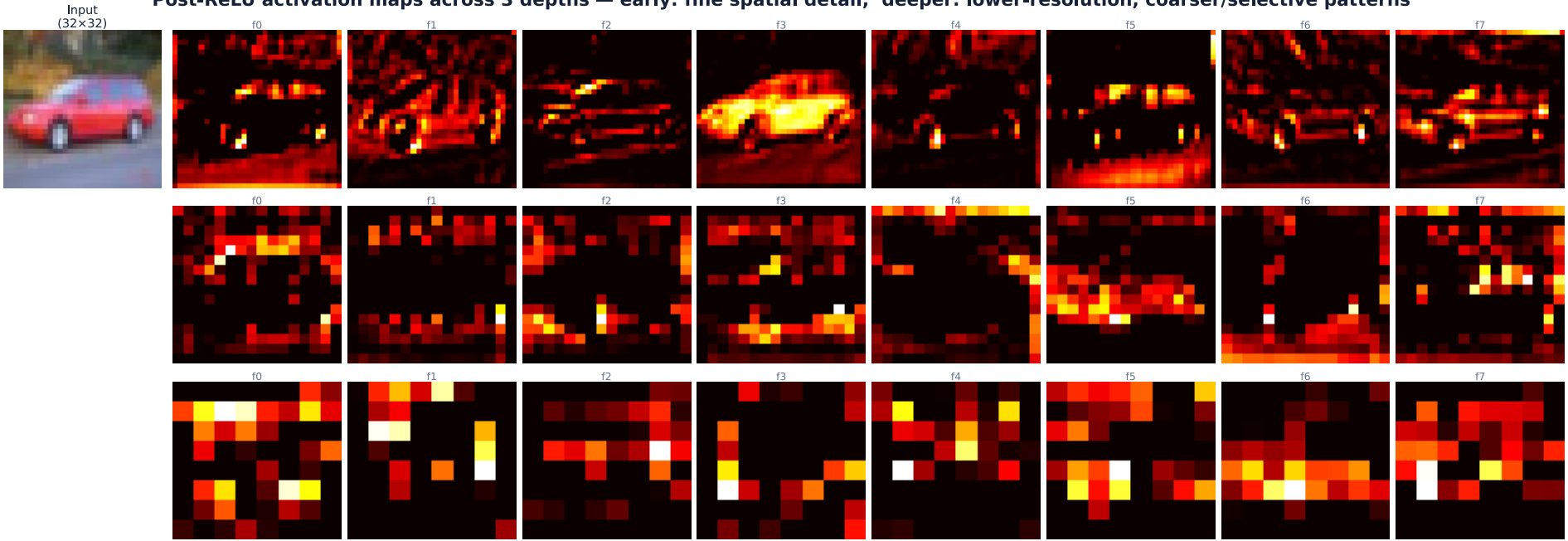
Same optimizer (AdamW), z-score normalization, and callbacks — the architecture is the key difference.

Feature Map Visualization — What Activations Do CNN Layers Produce?

What are feature maps?

The output of each Conv2D layer: shape (H, W, C). Each of the C channels is one feature map. Early activations: fine spatial detail (edges, colours). Deeper: lower-resolution, coarser, more class-relevant.

Post-ReLU activation maps across 3 depths — early: fine spatial detail; deeper: lower-resolution, coarser/selective patterns



Top-to-bottom: relu1 (32x32x32), relu2 (16x16x64), relu3 (8x8x128)

Feature Map Extraction — Keras Code

Build a single model to extract all three post-ReLU activation maps in one forward pass — efficient and avoids retracing warnings.

```
# Extract all three post-ReLU activation maps in one pass
feat_model = tf.keras.Model(
    inputs=cnn.input,
    outputs=[
        cnn.get_layer("relu1").output,  # 32×32×32
        cnn.get_layer("relu2").output,  # 16×16×64
        cnn.get_layer("relu3").output,  #  8×8×128
    ],
)

idx = img_idx
fmaps1, fmaps2, fmaps3 = feat_model(
    x_te[idx:idx+1], training=False
)

# Plotting code is in the notebook:
# show input + first 8 channels from each layer.
```

Key idea: feat_model shares weights with CNN — no retraining. Use **training=False** for inference-mode BN/Dropout.

QUICK QUIZ

A 3-block CNN is trained on CIFAR-10. Train accuracy = 97%, val accuracy = 63%. No augmentation, no Dropout. What is the most likely cause and best first fix?

A The CNN is underfitting — add more Conv2D layers and more filters to increase capacity.

B Severe overfitting — add Dropout after MaxPool + data augmentation (flip, translate) in the L6 pipeline.

C The learning rate is too high — reduce to 1e-5 to stabilize training loss.

D GlobalAveragePooling loses spatial information — replace with Flatten + Dense(256).

ANSWER

Answer: B — Severe overfitting; add Dropout + augmentation

Train=97% and val=63% is classic memorization, not insufficient capacity. Two complementary fixes:

- Mild Dropout: try 0.2–0.4 after later MaxPool blocks; randomly drops activations during training.
- Light augmentation: RandomFlip and small RandomTranslation; tune carefully because too much regularization can reduce accuracy.

Why not A: Train accuracy is already 97% — the model memorizes, it does not underfit. More layers worsen overfitting.

Why not C: LR affects convergence speed, not the train/val gap. The gap is a generalization problem, not instability.

Why not D: GAP is more regularizing than Flatten+Dense. It adds zero extra parameters.

Key Takeaways

Lecture 7 Summary

Local Connectivity

Shared kernel weights, not one weight per (input, output) pair. K×K×C_in×C_out params per layer.

Output Dimensions

floor((H+2P−K)/S)+1 (valid: P=0; same: output≈ceil(H/S)). Always verify before stacking layers.

Channels & Feature Maps

Each filter detects a different pattern. Early layers: edges. Deeper layers: textures, shapes.

Receptive Field

Grows with depth. K=3,S=1 adds 2 per layer; stride or pooling increases spacing between outputs, covering more input per layer.

Practical Default

Conv2D(F,3,padding='same')→BN→ReLU→MaxPool(2). Repeat×3, double filters. Then GAP→Dense(10).